

Rendimiento Maximo

NVIDIA Jetson AGX Orin 64GB

Guia Paso a Paso | JetPack 6.2.2 | Ubuntu 22.04 LTS

ESPECIFICACIONES DEL SISTEMA

Hardware: NVIDIA Jetson AGX Orin Developer Kit 64GB
SoC: Tegra 234 | CPU: 12x ARM Cortex-A78AE @ 2.2 GHz
GPU: Ampere 2048 CUDA + 64 Tensor Cores | 275 TOPS (INT8)
RAM: 64GB LPDDR5 @ 204.8 GB/s | CUDA 12.6 | cuDNN 9.3
TensorRT 10.3 | OpenCV 4.8 | 2x NVDLA v2.0

CONTENIDO DEL TUTORIAL

FASE 1	Modo de Potencia MAXN y Bloqueo de Relojes
FASE 2	Gestion de Memoria: Swap y ZRAM
FASE 3	Control Termico y Ventilador
FASE 4	Herramientas de Monitoreo (jtop / tegrastats)
FASE 5	Docker y Contenedores con GPU
FASE 6	Benchmarking y Validacion de Rendimiento
FASE 7	Optimizaciones Avanzadas (DLA, TensorRT)
APENDICE	Referencia Rapida de Comandos

FASE 1: Modo de Potencia MAXN y Bloqueo de Relojes

El Jetson AGX Orin 64GB esta clasificado para alcanzar 275 TOPS de rendimiento en operaciones INT8, pero por defecto opera en un modo conservador que apenas entrega una fraccion de ese potencial. La GPU funciona a ~600 MHz en lugar de sus 1300 MHz maximos, y la inferencia de un modelo LLM de 7B parametros apenas alcanza ~8 tokens/seg. Esta fase te guiara para desbloquear el 100% del rendimiento de tu hardware.¹

Por que es importante

Configuracion	GPU	LLM 7B (tokens/seg)	Diferencia
Sin MAXN (defecto)	~600 MHz	~8 tok/s	Linea base
MAXN + jetson_clocks	1300 MHz	~18-25 tok/s	~3x mas rapido

¹ [NVIDIA Jetson Developer Guide - Power & Performance](#)

Paso 1: Verificar el modo de potencia actual

Abre una terminal en tu Jetson y ejecuta el siguiente comando para ver en que modo de potencia se encuentra actualmente tu dispositivo:

```
sudo nvpmode l -q
```

La salida mostrara algo como:

```
NVPM WARN: ...
NV Power Mode: MAXN
0
```

El numero al final (0, 1, 2 o 3) es el ID del modo activo. Si no es 0 (MAXN), necesitas cambiarlo en el siguiente paso.

Paso 2: Ver todos los modos de potencia disponibles

Tu Jetson AGX Orin 64GB tiene 4 modos de potencia predefinidos. Puedes verlos ejecutando:

```
sudo nvpmode l -q --verbose | grep -A1 "MODE_NAME"
```

Tabla de Modos de Potencia – Jetson AGX Orin 64GB (JetPack 6.2)

ID	Nombre	TDP	CPUs	GPU Max	EMC Max	DLA Max
0	MAXN	Sin limite (~60W)	12	1300 MHz	3199 MHz	1600 MHz
1	MODE_15W	15W	4	420 MHz	2133 MHz	614 MHz

ID	Nombre	TDP	CPUs	GPU Max	EMC Max	DLA Max
2	MODE_30W	30W	8	624 MHz	3200 MHz	1369 MHz
3	MODE_50W	50W	12	828 MHz	3200 MHz	1369 MHz

IMPORTANTE

El modo MAXN (ID 0) no tiene limite de potencia y habilita los 12 nucleos CPU, la GPU a 1300 MHz y la memoria LPDDR5 a su maxima frecuencia. Usalo siempre para cargas de trabajo de IA.

Paso 3: Cambiar al modo MAXN

Ejecuta el siguiente comando para activar el modo de maxima potencia:

```
sudo nvpmode1 -m 0
```

Este cambio sobrevive a reinicios porque se escribe en el archivo `/etc/nvpmode1.conf`. Solo necesitas ejecutarlo una vez.²

Paso 4: Activar los 12 nucleos del CPU

Por defecto, algunos nucleos del CPU pueden estar desactivados. Antes de bloquear los relojes, asegurate de que los 12 nucleos esten activos:³

```
# Verificar cuantos CPUs estan online
cat /sys/devices/system/cpu/online

# Activar los nucleos que podrian estar apagados (4-7)
sudo su
echo 1 > /sys/devices/system/cpu/cpu4/online
echo 1 > /sys/devices/system/cpu/cpu5/online
echo 1 > /sys/devices/system/cpu/cpu6/online
echo 1 > /sys/devices/system/cpu/cpu7/online
exit

# Confirmar que todos estan activos
cat /sys/devices/system/cpu/online
# Salida esperada: 0-11
```

Paso 5: Bloquear relojes al maximo (jetson_clocks)

El comando `jetson_clocks` fija las frecuencias del CPU, GPU y bus de memoria EMC a sus valores maximos, deshabilitando el escalado dinamico de voltaje y frecuencia (DVFS):

```
sudo jetson_clocks
```

ATENCION

Este cambio es temporal y se pierde al reiniciar. En el paso siguiente lo haremos permanente.

Paso 6: Verificar la configuracion

Confirma que todo esta correctamente configurado ejecutando:

```
# Confirmar modo de potencia
sudo nvpmode1 -q

# Ver frecuencias actuales de CPU, GPU y EMC
sudo jetson_clocks --show
```

En la salida de `jetson_clocks --show`, busca que `CurrentFreq` sea igual a `MaxFreq` en todas las lineas:

```
CPU Cluster Switching: Disabled
cpu0: Online=1 Governor=schedutil MinFreq=729600
MaxFreq=2201600 CurrentFreq=2201600
...
GPU MinFreq=306000000 MaxFreq=1300500000
CurrentFreq=1300500000
EMC MinFreq=204000000 MaxFreq=3199000000
CurrentFreq=3199000000
```

CONSEJO

Si `CurrentFreq` no es igual a `MaxFreq`, ejecuta de nuevo `sudo jetson_clocks` y verifica.

Paso 7: Hacer jetson_clocks permanente (arranque automatico)

Crea un servicio systemd para que los relojes se bloqueen automaticamente en cada arranque:

```
# Intentar habilitar el servicio existente
sudo systemctl enable jetson_clocks 2>/dev/null

# Si no existe, crear el servicio manualmente:
sudo tee /etc/systemd/system/jetson_clocks.service > /dev/null << 'EOF'
[Unit]
Description=Lock Jetson clocks at maximum frequency
After=multi-user.target

[Service]
Type=oneshot
ExecStartPre=/usr/sbin/nvpmode1 -m 0
ExecStart=/usr/bin/jetson_clocks
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF

# Activar e iniciar el servicio
sudo systemctl daemon-reload
sudo systemctl enable jetson_clocks
sudo systemctl start jetson_clocks

# Verificar que esta activo
sudo systemctl status jetson_clocks
```

² [NVIDIA Forums - Diferencias MAXN via flash vs nvpmoel](#)

³ [RidgeRun - Maximizing Jetson Orin Performance](#)

FASE 2: Gestion de Memoria – Swap y ZRAM

El Jetson AGX Orin 64GB tiene una arquitectura de memoria unificada: los 64 GB de LPDDR5 son compartidos entre el CPU y la GPU. Esto significa que al cargar un modelo LLM grande (por ejemplo, 70B parametros), la memoria se consume tanto por el modelo como por el sistema operativo. Configurar correctamente el swap es critico para evitar bloqueos y poder cargar modelos grandes.⁴

Arquitectura de Memoria del AGX Orin 64GB

Componente	Valor	Descripcion
RAM Total	64 GB LPDDR5	Compartida CPU + GPU (iGPU)
Ancho de Banda	204.8 GB/s	Bus de 256 bits a 3200 MHz
GPU VRAM	Compartida	No tiene VRAM dedicada, usa la RAM principal
Bus EMC	3199 MHz (MAXN)	Controlador de memoria externa

Paso 1: Verificar el estado actual de la memoria

```
# Ver memoria RAM y swap actual
free -h

# Ver informacion detallada de swap
swapon --show

# Ver ZRAM configurada (por defecto en JetPack)
zramctl
```

Paso 2: Entender ZRAM vs Swap en disco

JetPack 6.2 configura ZRAM por defecto como swap comprimida en RAM. Esto es util para uso general pero tiene limitaciones para cargas de IA pesadas:

Tipo	Velocidad	Capacidad	Recomendado para
ZRAM (RAM comprimida)	Muy rapida	Limitada (~50% RAM)	Uso general, apps ligeras
Swap en NVMe SSD	Rapida	Configurable (16-64 GB)	LLM 70B, modelos grandes
Swap en SD Card	Lenta	Configurable	NO recomendado (desgaste)

Paso 3: Desactivar ZRAM (recomendado para LLM)

Para modelos grandes, es mejor desactivar ZRAM y usar swap en disco NVMe. ZRAM consume RAM real (comprimida) que necesitas para el modelo:⁴

```
# Desactivar todos los dispositivos ZRAM
sudo swapoff -a

# Deshabilitar ZRAM en el arranque
sudo systemctl disable nvzramconfig 2>/dev/null || true

# Verificar que ZRAM esta desactivada
zramctl

# No deberia mostrar nada
```

Paso 4: Crear archivo Swap en NVMe SSD

Crea un archivo de swap de 32 GB en tu disco NVMe (ajusta el tamaño según necesites):

```
# Crear archivo de swap de 32 GB
sudo fallocate -l 32G /mnt/swap/swapfile

# Si fallocate no funciona, usar dd:
# sudo dd if=/dev/zero of=/mnt/swap/swapfile bs=1G count=32

# Establecer permisos correctos (solo root)
sudo chmod 600 /mnt/swap/swapfile

# Formatear como swap
sudo mkswap /mnt/swap/swapfile

# Activar el swap
sudo swapon /mnt/swap/swapfile

# Verificar
swapon --show
free -h
```

NOTA

Si tu sistema no tiene `/mnt/swap/`, primero crea el directorio: `sudo mkdir -p /mnt/swap`. Si tu NVMe esta montado en otra ruta, ajusta la ruta del swapfile.

Paso 5: Hacer el swap permanente

Agrega la línea al archivo `fstab` para que el swap se active automáticamente en cada arranque:

```
# Agregar entrada a fstab
echo "/mnt/swap/swapfile none swap sw 0 0" | sudo tee -a /etc/fstab

# Verificar que fstab es correcto
cat /etc/fstab | grep swap
```

Paso 6: Ajustar swappiness para rendimiento de IA

El parametro `swappiness` controla la tendencia del kernel a mover paginas de memoria al swap. Para IA, queremos mantener los datos en RAM el mayor tiempo posible:

```
# Ver valor actual
cat /proc/sys/vm/swappiness

# Establecer valor bajo (1 = solo usar swap en emergencia)
sudo sysctl vm.swappiness=1

# Hacerlo permanente
echo "vm.swappiness=1" | sudo tee -a /etc/sysctl.conf

# Aplicar cambios
sudo sysctl -p
```

CONSEJO

Un `swappiness=1` le dice al kernel que solo use swap cuando sea absolutamente necesario. Esto mantiene los pesos del modelo en RAM fisica, maximizando la velocidad de inferencia.

Paso 7: Liberar memoria del escritorio (recomendado)

Si usas tu Jetson principalmente para cargas de IA, desactivar el entorno grafico libera ~1.5 GB de RAM:⁵

```
# Desactivar el escritorio grafico
sudo systemctl set-default multi-user.target

# Reiniciar para aplicar
sudo reboot

# Para restaurar el escritorio grafico:
# sudo systemctl set-default graphical.target
# sudo reboot
```

⁴ [NVIDIA Forums - ZRAM swap en Jetson Orin](#)

⁵ [Hackster.io - Running LLMs with TensorRT-LLM on Jetson AGX Orin](#)

FASE 3: Control Termico y Ventilador

El modo MAXN genera mas calor. Tu Jetson AGX Orin tiene un sistema de refrigeracion activa con ventilador que se ajusta automaticamente, pero puedes optimizarlo para mantener el maximo rendimiento bajo cargas sostenidas de IA.⁶

Zonas Termicas y Umbrales

Zona Termica	Modulo	Throttling SW	Throttling HW	Apagado
CPU-therm	Procesador ARM	99 C	103 C	105 C
GPU-therm	GPU Ampere	99 C	103 C	105 C
SOC0-therm	SoC General	99 C	103 C	105 C
CV0-therm	Computer Vision	99 C	103 C	105 C
Tboard_tegra	Placa (board)	N/A	N/A	N/A

Temperaturas Normales de Operacion

Componente	Normal (idle)	Carga IA sostenida	Peligro
GPU	35-45 C	55-80 C	> 85 C
CPU	30-42 C	50-75 C	> 85 C
Board	30-50 C	45-65 C	> 75 C

Paso 1: Verificar las temperaturas actuales

```
# Leer todas las zonas termicas
for zone in /sys/class/thermal/thermal_zone*/*; do
type=$(cat ${zone}type)
temp=$(cat ${zone}temp)
echo "${type}: $((temp/1000)) C"
done
```

Paso 2: Configurar el perfil del ventilador

El Jetson tiene dos perfiles de ventilador: quiet (silencioso) y cool (refrigeracion). Para cargas de IA, usa el perfil cool:

```
# Ver perfil actual del ventilador
sudo cat /etc/nvfancontrol.conf | grep FAN_DEFAULT_PROFILE

# Para cambiar a 'cool' (maxima refrigeracion):
# Editar /etc/nvfancontrol.conf y cambiar la linea:
# FAN_DEFAULT_PROFILE cool
```



```
sudo sed -i 's/FAN_DEFAULT_PROFILE quiet/FAN_DEFAULT_PROFILE cool/' \
/etc/nvfancontrol.conf

# Reiniciar el servicio de control del ventilador
sudo systemctl restart nvfancontrol
```

Perfil	Comportamiento	Uso recomendado
quiet	Ventilador a baja velocidad, prioriza silencio	Desarrollo ligero, idle
cool	Ventilador a mayor velocidad, prioriza temperatura	Inferencia IA, cargas pesadas

Paso 3: Forzar ventilador al maximo (manual)

Para sesiones de inferencia intensiva donde necesitas la maxima refrigeracion:

```
# Detener el control automatico del ventilador
sudo systemctl stop nvfancontrol

# Establecer el ventilador al 100% (PWM = 255)
echo 255 | sudo tee /sys/devices/pwm-fan/target_pwm

# Para restaurar el control automatico:
sudo systemctl start nvfancontrol
```

ADVERTENCIA

Forzar el ventilador al maximo genera mas ruido. Usa este modo solo durante sesiones de inferencia prolongadas o benchmarks. Recuerda restaurar el control automatico al terminar.

⁶ [NVIDIA Docs - Thermal Cooling for Jetson Orin](#)

FASE 4: Herramientas de Monitoreo

Monitorear el rendimiento en tiempo real es esencial para verificar que las optimizaciones estan funcionando correctamente y para detectar problemas de throttling termico.⁷

Paso 1: Instalar y usar jtop (recomendado)

jtop es el dashboard interactivo mas completo para monitorear tu Jetson. Muestra CPU, GPU, RAM, temperaturas, frecuencias, potencia y mas en tiempo real:

```
# Instalar jetson-stats (incluye jtop)
sudo apt update
sudo apt install -y python3-pip
sudo pip3 install -U jetson-stats

# IMPORTANTE: Reiniciar despues de instalar
sudo reboot

# Despues del reinicio, ejecutar:
jtop
```

Tabs de jtop

Tab	Que muestra	Para que sirve
1 - GPU	Uso GPU, frecuencia, temperatura	Verificar que GPU opera a 1300 MHz
2 - CPU	Carga por nucleo, frecuencias, governor	Confirmar 12 nucleos activos a 2.2 GHz
3 - MEM	RAM usada/libre, swap, ZRAM	Monitorear uso de memoria durante inferencia
4 - ENG	DLA, PVA, motores de video	Verificar uso de aceleradores
5 - CTRL	nvpmodel, jetson_clocks, fan	Control interactivo de potencia y ventilador
6 - INFO	JetPack, L4T, CUDA, cuDNN	Verificar versiones de software

CONSEJO

Desde la tab CTRL de jtop puedes cambiar el modo de potencia, activar jetson_clocks y ajustar el ventilador de forma interactiva, sin comandos.

Paso 2: Usar tegrastats (linea de comandos)

tegrastats viene preinstalado y muestra estadisticas de rendimiento en texto plano. Es ideal para monitorear en una terminal remota o en scripts:

```
# Ejecutar tegrastats con actualizacion cada segundo
sudo tegrastats --interval 1000

# Guardar log en archivo (util para analisis)
```

```
sudo tegrastats --interval 1000 --logfile /tmp/tegrastats.log
```

Como leer la salida de tegrastats

Campo	Ejemplo	Significado
RAM	7467/62827MB	RAM usada / total en MB
SWAP	0/32768MB	Swap usada / total en MB
CPU	[45%@2201,38%@2201,...]	Uso% y frecuencia por nucleo
GR3D_FREQ	99%@1300	Uso% GPU y frecuencia en MHz
EMC_FREQ	98%@3199	Uso% memoria y frecuencia EMC
GPU@55C	GPU@55C	Temperatura de la GPU en grados C
CPU@52C	CPU@52C	Temperatura del CPU en grados C
POM_5V_GPU	12500/15000	Potencia GPU actual/promedio (mW)

NOTA

Si ves GR3D_FREQ por debajo de 1300 MHz durante inferencia, puede indicar throttling termico o que jetson_clocks no esta activo. Revisa las temperaturas.

Paso 3: Script de monitoreo personalizado

Crea un script practico para monitorear durante sesiones de inferencia:

```
#!/bin/bash
# monitor_jetson.sh - Monitoreo continuo
echo "=== Jetson AGX Orin 64GB - Monitor ==="
echo "Modo de potencia:"
sudo nvpmode -q 2>/dev/null | tail -2
echo ""
echo "Frecuencias actuales:"
sudo jetson_clocks --show 2>/dev/null | \
grep -E "CurrentFreq|MinFreq|MaxFreq"
echo ""
echo "Temperaturas:"
for zone in /sys/class/thermal/thermal_zone*; do
type=$(cat ${zone}type 2>/dev/null)
temp=$(cat ${zone}temp 2>/dev/null)
if [ -n "$temp" ]; then
echo " ${type}: $((temp/1000))C"
fi
done
echo ""
echo "Memoria:"
free -h | head -3
```

```
# Guardar y ejecutar
chmod +x monitor_jetson.sh
./monitor_jetson.sh
```

⁷ [JetsonHacks - jtop: The Ultimate Tool for Monitoring NVIDIA Jetson](#)

FASE 5: Docker y Contenedores con GPU

Los contenedores Docker son la forma preferida de ejecutar cargas de trabajo de IA en Jetson, ya que garantizan compatibilidad de bibliotecas y aislamiento. El runtime NVIDIA permite acceder a la GPU desde dentro del contenedor.⁸

Paso 1: Instalar Docker y NVIDIA Container Toolkit

```
# Instalar Docker
sudo apt update
sudo apt install -y docker.io

# Agregar tu usuario al grupo docker
sudo usermod -aG docker $USER

# Instalar NVIDIA Container Toolkit
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L
https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
https://#g' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt update
sudo apt install -y nvidia-container-toolkit
```

Paso 2: Configurar Docker para usar GPU por defecto

```
# Configurar el runtime NVIDIA
sudo nvidia-ctk runtime configure --runtime=docker

# Agregar runtime por defecto
sudo jq '. + {"default-runtime": "nvidia"}' \
/etc/docker/daemon.json | \
sudo tee /etc/docker/daemon.json.tmp && \
sudo mv /etc/docker/daemon.json.tmp /etc/docker/daemon.json

# Reiniciar Docker
sudo systemctl daemon-reload
sudo systemctl restart docker

# Aplicar cambios de grupo (o cerrar sesion y volver a entrar)
newgrp docker
```

Paso 3: Verificar acceso a GPU desde Docker

```
# Prueba rapida: nvidia-smi dentro del contenedor
docker run --rm --runtime nvidia \
nvcr.io/nvidia/l4t-base:r36.2.0 \
```

```
nvidia-smi

# Si no funciona, prueba con la imagen base:
docker run --rm --runtime nvidia \
--gpus all \
nvcr.io/nvidia/l4t-base:r36.2.0 \
cat /etc/nv_tegra_release
```

IMPORTANTE

Siempre usa imagenes base l4t-base o l4t-cuda de NVIDIA como base para tus contenedores Jetson. Las imagenes Docker estandar de x86 no son compatibles con la GPU de Jetson (arquitectura ARM64 + Tegra).

Paso 4: Parametros optimos para docker run

Usa estos flags al ejecutar contenedores de IA:

```
docker run --rm --runtime nvidia \
--network=host \
--shm-size=8g \
-v /tmp/argus_socket:/tmp/argus_socket \
-v /etc/enctune.conf:/etc/enctune.conf \
-v /etc/nv_tegra_release:/etc/nv_tegra_release \
-v /run/jtop.sock:/run/jtop.sock \
-v $HOME/data:/data \
<imagen>
```

Parametro	Funcion
--runtime nvidia	Habilita acceso a GPU NVIDIA dentro del contenedor
--network=host	Usa la red del host (evita problemas de puertos)
--shm-size=8g	Aumenta memoria compartida (requerido por PyTorch)
-v /run/jtop.sock:...	Permite usar jtop desde dentro del contenedor
-v \$HOME/data:/data	Monta directorio local para modelos y datos

⁸ [NVIDIA Forums - Running AI Docker containers on Jetson with GPU](#)

FASE 6: Benchmarking y Validacion de Rendimiento

Despues de aplicar todas las optimizaciones, es crucial validar que el rendimiento real coincide con el esperado. Estos benchmarks te permiten medir y comparar.

Paso 1: Benchmark de GPU con CUDA

Ejecuta las muestras de CUDA que vienen preinstaladas con JetPack para verificar el rendimiento basico de la GPU:

```
# Ubicacion de samples CUDA
ls /usr/local/cuda/samples/ 2>/dev/null || \
ls /usr/local/cuda-12.6/extras/demo_suite/

# Ejecutar bandwidthTest (mide ancho de banda de memoria)
/usr/local/cuda-12.6/extras/demo_suite/bandwidthTest

# Resultado esperado en MAXN:
# Host to Device: ~25-30 GB/s
# Device to Host: ~25-30 GB/s
# Device to Device: ~180-200 GB/s
```

Paso 2: Benchmark de LLM con Ollama

Si tienes Ollama instalado, mide la velocidad de inferencia de un modelo:

```
# Benchmark con modelo 3B (rapido)
ollama run llama3.2 --verbose \
"Escribe una historia de 100 palabras" 2>&1 | \
grep -E "eval rate|tokens/s"

# Resultados esperados en MAXN (llama3.2 3B Q4_K_M):
# eval rate: 25-40 tokens/seg

# Benchmark con modelo 7-8B
ollama run llama3.1:8b --verbose \
"Explain quantum computing in 50 words" 2>&1 | \
grep -E "eval rate|tokens/s"

# Resultados esperados en MAXN (8B Q4_K_M):
# eval rate: 18-25 tokens/seg
```

Paso 3: Benchmark con llama.cpp (nativo)

Si compilaste llama.cpp con CUDA para tu Jetson:

```
# Benchmark con llama-bench (si esta compilado)
./llama-bench \
-m /ruta/al/modelo.gguf \
-ngl 99 \
-t 12 \
--flash-attn on
```

```
# 0 con llama-cli en modo single-turn
time llama-cli \
-m /ruta/al/modelo.gguf \
-ngl 99 -c 4096 -t 12 \
--flash-attn on \
--prompt "Escribe un poema corto" \
--single-turn
```

Paso 4: Tabla de rendimiento esperado

Rendimiento aproximado en modo MAXN con modelos cuantizados Q4_K_M:

Modelo	Tamano	VRAM	Tokens/seg	Herramienta
Llama 3.2 3B	Q4_K_M (~2 GB)	~3 GB	30-45 tok/s	Ollama / llama.cpp
Llama 3.1 8B	Q4_K_M (~5 GB)	~6 GB	18-28 tok/s	Ollama / llama.cpp
Qwen 2.5 14B	Q4_K_M (~9 GB)	~10 GB	12-18 tok/s	Ollama / llama.cpp
Qwen 3.5 27B	Q4_K_XL (~17 GB)	~19 GB	8-14 tok/s	llama.cpp
Llama 3.1 70B	Q4_K_M (~40 GB)	~42 GB	3-6 tok/s	llama.cpp + swap

CONSEJO

Los tokens/seg dependen del contexto, longitud de prompt, y si el modelo cabe enteramente en la memoria unificada. Modelos que requieren swap seran significativamente mas lentos.

FASE 7: Optimizaciones Avanzadas

7.1 Deep Learning Accelerator (DLA)

El Jetson AGX Orin 64GB incluye 2 nucleos DLA v2.0 (Deep Learning Accelerator) que pueden ejecutar modelos de vision en paralelo con la GPU, liberando recursos GPU para otras tareas. El DLA opera en INT8 y es 3-5x mas eficiente en energia que la GPU para cargas compatibles.⁹

Propiedad	Valor en MAXN
Nucleos DLA	2 (DLA0 y DLA1)
Frecuencia maxima DLA Core	1600 MHz
Frecuencia maxima DLA Falcon	844.8 MHz
Precision soportada	INT8, FP16
Eficiencia energetica	3-5x mejor que GPU (por watt)

```
# Ejecutar modelo en DLA con trtexec
trtexec --onnx=model.onnx \
--int8 \
--useDLACore=0 \
--allowGPUFallback \
--saveEngine=model_dla.engine
```

7.2 TensorRT para Inferencia Optimizada

TensorRT 10.3 (incluido en JetPack 6.2.2) permite convertir modelos a engines optimizados con hasta 2-4x mejor rendimiento que frameworks genericos.¹⁰

```
# Convertir modelo ONNX a TensorRT engine
trtexec --onnx=model.onnx \
--fp16 \
--workspace=4096 \
--saveEngine=model_fp16.engine

# Con INT8 (requiere calibracion)
trtexec --onnx=model.onnx \
--int8 \
--calib=calibration_cache.bin \
--saveEngine=model_int8.engine
```

7.3 TensorRT-LLM para Modelos de Lenguaje

TensorRT-LLM optimiza especificamente la inferencia de LLM en GPU NVIDIA. Reduce el uso de memoria pico y mejora la utilizacion de la GPU:¹⁰

```
# Flujo completo TensorRT-LLM para Llama 8B

# 1. Convertir checkpoint de HuggingFace
python convert_checkpoint.py \
--model_dir ./Llama-3.1-8B-Instruct \
--output_dir ./Llama-8B-convert \
--dtype float16

# 2. Construir engine TensorRT
trtllm-build \
--checkpoint_dir ./Llama-8B-convert \
--gpt_attention_plugin float16 \
--gemm_plugin float16 \
--output_dir ./Llama-8B-engine

# 3. Ejecutar inferencia
python run.py \
--engine_dir ./Llama-8B-engine \
--max_output_len 100 \
--tokenizer_dir ./Llama-3.1-8B-Instruct \
--input_text "Hello, world" \
--gpu_weights_percent 70
```

7.4 AWQ Cuantizacion para Modelos Grandes

AWQ (Activation-aware Weight Quantization) permite ejecutar modelos mucho mas grandes al reducir los pesos a INT4, con perdida minima de calidad:

Modelo 70B	FP16	INT8	INT4 (AWQ)
Memoria requerida	~140 GB	~70 GB	~35 GB
Compatible con 64GB	No	Si (ajustado)	Si (comodo)

⁹ [NVIDIA Blog - Maximizing DL Performance on Jetson Orin with DLA](#)
¹⁰ [Hackster.io - Running LLMs with TensorRT-LLM on Jetson AGX Orin](#)

APENDICE: Referencia Rapida de Comandos

Comandos Esenciales

Accion	Comando
Activar modo MAXN	sudo nvpmodel -m 0
Bloquear relojes al maximo	sudo jetson_clocks
Ver modo de potencia actual	sudo nvpmodel -q
Ver frecuencias actuales	sudo jetson_clocks --show
Monitoreo interactivo	jtop
Monitoreo de texto	sudo tegrastats --interval 1000
Ver temperaturas	cat /sys/class/thermal/thermal_zone*/temp
Ventilador al maximo	echo 255 sudo tee /sys/devices/pwm-fan/target_pwm
Ver memoria	free -h
Ver swap	swapon --show
Ver nucleos CPU activos	cat /sys/devices/system/cpu/online
Desactivar escritorio	sudo systemctl set-default multi-user.target
Reiniciar Docker	sudo systemctl restart docker
Probar GPU en Docker	docker run --rm --runtime nvidia nvcr.io/nvidia/l4t-base:r36.2.0 nvidia-smi

Secuencia Completa de Optimizacion (resumen)

Ejecuta estos comandos en orden para configurar tu Jetson AGX Orin 64GB al maximo rendimiento:

```
# ===== CONFIGURACION INICIAL (una sola vez) =====

# 1. Modo MAXN
sudo nvpmodel -m 0

# 2. Activar todos los CPU cores
sudo su -c '
for cpu in 4 5 6 7; do
echo 1 > /sys/devices/system/cpu/cpu${cpu}/online
done
'

# 3. Bloquear relojes
sudo jetson_clocks

# 4. Crear servicio jetson_clocks permanente
```

```
sudo tee /etc/systemd/system/jetson_clocks.service > /dev/null << 'EOF'
[Unit]
Description=Lock Jetson clocks at maximum frequency
After=multi-user.target
[Service]
Type=oneshot
ExecStartPre=/usr/sbin/nvpmode -m 0
ExecStart=/usr/bin/jetson_clocks
RemainAfterExit=yes
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable jetson_clocks

# 5. Desactivar ZRAM y crear swap en NVMe
sudo systemctl disable nvzramconfig 2>/dev/null || true
sudo swapoff -a
sudo mkdir -p /mnt/swap
sudo fallocate -l 32G /mnt/swap/swapfile
sudo chmod 600 /mnt/swap/swapfile
sudo mkswap /mnt/swap/swapfile
sudo swapon /mnt/swap/swapfile
echo "/mnt/swap/swapfile none swap sw 0 0" | sudo tee -a /etc/fstab

# 6. Ajustar swappiness
echo "vm.swappiness=1" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# 7. Ventilador en perfil cool
sudo sed -i 's/FAN_DEFAULT_PROFILE quiet/FAN_DEFAULT_PROFILE cool/' \
/etc/nvfancontrol.conf
sudo systemctl restart nvfancontrol

# 8. Desactivar escritorio (opcional)
sudo systemctl set-default multi-user.target

# 9. Instalar herramientas de monitoreo
sudo pip3 install -U jetson-stats

# 10. Reiniciar
sudo reboot
```

Solucion de Problemas Comunes

Problema	Causa Probable	Solucion
GPU frecuencia baja durante inferencia	jetson_clocks no activo o throttling termico	Ejecutar: sudo jetson_clocks y verificar temperaturas
RAM insuficiente para modelo 70B	Sin swap o swap insuficiente	Crear swap de 32-64 GB en NVMe SSD
Docker: runtime nvidia not found	nvidia-container-toolkit no instalado	Instalar toolkit y reiniciar Docker

Problema	Causa Probable	Solucion
Temperaturas > 85C sostenidas	Ventilacion bloqueada o perfil quiet	Cambiar a perfil cool, verificar ventilacion
jtop no detecta JetPack	Version de jetson-stats desactualizada	Actualizar: sudo pip3 install -U jetson-stats
EMC frecuencia en 204 MHz	DVFS activo o modo de potencia bajo	Verificar nvpmode -m 0 y jetson_clocks

Recursos y Enlaces Utiles

- 1. [NVIDIA Jetson Developer Guide - Power & Performance](#)
- 2. [NVIDIA Blog - JetPack 6.2 Super Mode](#)
- 3. [NVIDIA Blog - Maximizing DL Performance with DLA](#)
- 4. [GitHub - jetson-stats \(jtop\)](#)
- 5. [RidgeRun - Jetson Orin Performance Tuning](#)
- 6. [Hackster.io - TensorRT-LLM on Jetson AGX Orin](#)